Stock Portfolio Risk & Forecasting Platform

A Django REST Framework backend combining JWT-secured, role-based portfolio management with a machine learning risk and forecast engine — built API-first for a future Flutter client.

Prepared by: Dion James Smith

Version 1.0

September 2026

Table of Contents

Table of Contents 2
1. Executive Summary 4
2. Project Objectives 4
3. Technology Stack 4
4. System Architecture Overview 5
4.1 Layers 5
4.2 Request Flow (synchronous) 5
4.3 Background Flow (asynchronous) 5
5. Application (Django App) Structure 6
5.1 accounts 6
5.2 portfolios 6
5.3 market_data 6
5.4 analytics 6
5.5 core 6
6. Database Design 6
6.1 Entity Overview 6
6.2 Table: User (accounts app) 7
6.3 Table: Portfolio (portfolios app) 7
6.4 Table: Holding (portfolios app) 7
6.5 Table: PriceHistory (market_data app) 8
6.6 Relationship Diagram (described) 8
6.7 Representative Join-Based Queries 8
7. Authentication & Authorization 9
7.1 Why a Custom User Model 9
7.2 JWT Token Lifecycle 9
7.3 Token Lifetimes 9
7.4 Authorization Model 9
7.5 Custom Permission Classes 10
8. Roles & Permission Matrix 10
9. API Endpoint Reference 10
9.1 accounts 11
9.2 portfolios 11
9.3 market_data 11
9.4 analytics 12
10. Serializers Overview 12

UserSerializer 12
RegisterSerializer 12
LoginSerializer 12
PortfolioSerializer 12
HoldingSerializer 13
PriceHistorySerializer 13
RiskSummarySerializer 13
List vs. detail field strategy 13
11. Celery & Redis Task Design 13
11.1 Redis's Three Roles 13
11.2 Scheduled and On-Demand Tasks 13
11.3 Caching Strategy 14
11.4 Rate Limiting 14
12. Machine Learning Pipeline 14
12.1 Data Sources 14
12.2 Feature Engineering 14
12.3 Model 1 — Risk Classification 15
12.4 Model 2 — Directional Forecast 15
12.5 Training Methodology 15
12.6 Evaluation Metrics 15
12.7 Retraining & Versioning 15
13. Environment Configuration Reference 15
13.1 Database 16
13.2 JWT 16
13.3 Redis & Celery 16
14. Project Directory Structure 16
15. Security & Non-Functional Considerations 17
16. Future Scope & Flutter Integration Readiness 17
17. Machine Learning Dataset Source 18
17.1 Why It Maps Directly onto the PriceHistory Table 18
17.2 Why 10 Years of Daily Data Specifically Fits This ML Design 18
18. Appendix — Sample Interaction Flow 18

1. Executive Summary

This project is a backend platform that lets investors track stock portfolios, view real market data, and receive machine-learning-driven risk classifications and short-term price direction forecasts for each holding. It is built with Django and Django REST Framework, secured with a custom JWT authentication scheme built on a purpose-made User model, and organised around three roles — investor, analyst, and admin — each with distinct data access rules.

Two supervised machine learning models sit at the core of the analytics layer: a Random Forest classifier that scores each holding's risk level, and a Gradient Boosting classifier that predicts the likely next-day price direction. Both are trained on technical indicators engineered from historical price data and retrained on a schedule using Celery.

Redis and Celery handle everything that should not block a web request: daily market data ingestion, nightly risk/forecast recomputation, periodic model retraining, and response caching for expensive aggregation endpoints. PostgreSQL is the system of record, accessed exclusively through the Django ORM.

The API is deliberately client-agnostic. Every feature is exposed through versioned REST endpoints with DRF serializers, so the same backend can be consumed by a web dashboard or, as planned in a later phase, a Flutter mobile application — without backend changes.

2. Project Objectives

This project is designed to demonstrate the following engineering competencies in a single, coherent system:

- **Custom authentication architecture —** a dedicated accounts app with its own User model, JWT issuing and refresh logic, and role-based, object-level authorization, rather than relying on Django's default User model or a third-party auth app out of the box.
- **Relational database design —** a normalized schema with clear primary key / foreign key relationships, including a self-referencing relationship, and queries that rely on Django ORM joins and aggregations rather than denormalized convenience fields.
- **Applied, explainable machine learning —** two distinct, well-understood algorithms (not a black-box deep learning model) integrated into a live request/response and background-task pipeline, with proper time-based validation.
- **Asynchronous system design —** Celery and Redis used for scheduled ingestion, scheduled recomputation, on-demand background tasks, and caching — not just as a checkbox integration.
- **API-first backend design —** every capability exposed through serializers and REST endpoints so a Flutter client (or any client) can be built against it later without touching backend logic.

3. Technology Stack

Layer	Technology	Purpose in this project
Web / API framework	Django + Django REST Framework	Application server, ORM, serializers, viewsets, routing

Layer	Technology	Purpose in this project
Database	PostgreSQL	System of record for users, portfolios, holdings, and price history
Data access	Django ORM	All reads and writes go through the ORM — no raw SQL
Authentication	Custom JWT implementation	Access/refresh token issuing and validation against a custom User model
Task queue	Celery	Scheduled and on-demand background processing
Broker / cache	Redis	Celery message broker, task result backend, and application-level response cache
Machine learning	scikit-learn (Random Forest, Gradient Boosting)	Risk classification and directional forecasting
Planned client	Flutter	Mobile client to be built against the existing API in a later phase

4. System Architecture Overview

The system is organised into five logical layers. Each layer has a single, clear responsibility, which keeps the ML and async pieces from leaking into the request/response cycle.

4.1 Layers

- **Client layer —** any HTTP client capable of sending a JWT-authenticated request; the current reference client is a browsable API / Postman, with a Flutter app planned as the primary consumer.
- **API layer —** Django REST Framework viewsets and routers, JWT authentication middleware, and permission classes that gate access before any business logic runs.
- **Business logic layer —** serializers (validation, shaping of input/output) and view-level logic (e.g., resolving "my portfolios" vs "assigned client portfolios" based on role).
- **Data layer —** PostgreSQL, accessed only through the Django ORM, with select_related / prefetch_related used to keep join-heavy endpoints efficient.
- **Asynchronous layer —** Celery workers and Celery Beat, backed by Redis, handling market data ingestion, ML inference, model retraining, and cache invalidation outside the request/response cycle.

4.2 Request Flow (synchronous)

A typical authenticated request flows: Client → JWT Authentication (validate access token) → Permission Classes (role + object-level checks) → ViewSet → Serializer (validate/shape data) → Django ORM → PostgreSQL → Serializer (shape response) → Client. Read-heavy aggregation endpoints check Redis for a cached response before touching the database.

4.3 Background Flow (asynchronous)

Celery Beat triggers scheduled jobs (daily price ingestion, nightly risk/forecast recomputation, periodic model retraining) that are placed on a Redis-backed queue and picked up by Celery workers. Workers write their results back to PostgreSQL through the ORM and invalidate any related Redis cache entries so the next read reflects fresh data.

5. Application (Django App) Structure

The project is split into focused Django apps, each owning a narrow slice of the domain. This mirrors how a production Django codebase is typically organised and keeps the custom User model isolated from business-domain apps.

5.1 accounts

Owns the custom User model and everything related to identity: registration, login, JWT issuing and refresh, profile management, and admin-driven role/analyst-assignment management. No other app defines a user-like model — every other app references accounts.User by foreign key.

5.2 portfolios

Owns the Portfolio and Holding models. Provides CRUD for a user's portfolios and holdings, and computed views such as portfolio value and holding-level detail (including the ML fields written by the analytics app).

5.3 market_data

Owns the PriceHistory model. Responsible for ingesting daily OHLCV (open/high/low/close/volume) data per ticker via Celery tasks, and exposing read endpoints for current price and historical series.

5.4 analytics

Holds the machine learning pipeline: feature engineering, model training/retraining scripts, and the Celery tasks that periodically compute each holding's risk_label, risk_score, predicted_direction, and prediction_confidence, writing the results back onto the Holding row. Exposes read-only endpoints that present these figures, plus a portfolio-level risk summary. This app intentionally owns no database table of its own — it computes and annotates data owned by portfolios.

5.5 core

A small shared app for cross-cutting concerns: custom permission classes (e.g., object ownership checks), pagination classes, standard exception handling, and Redis-backed throttle classes. Kept separate so business apps do not duplicate this logic.

6. Database Design

The schema is intentionally kept to four core tables, each with a clear ownership boundary. Every foreign key enforces a real one-to-many relationship; there is also one self-referencing foreign key used to model analyst-to-investor assignment without a separate join table.

6.1 Entity Overview

Table	Owning app	Represents
User	accounts	Every person in the system — investor, analyst, or admin
Portfolio	portfolios	A named collection of holdings belonging to one investor
Holding	portfolios	A single stock position within a portfolio, plus its latest ML outputs
PriceHistory	market_data	Daily OHLCV market data for a ticker, shared across all holdings of that ticker

6.2 Table: User (accounts app)

Field	Type	Constraints / Notes
id	Integer	Primary key
username	String	Unique, required
email	String	Unique, required
password	String (hashed)	Never stored or transmitted in plain text
role	Enum	investor / analyst / admin
assigned_analyst_id	Integer	Self-referencing foreign key to User.id; nullable; set only for investors, points at the analyst responsible for them
is_active	Boolean	Standard account-enabled flag
date_joined	DateTime	Set on creation
updated_at	DateTime	Updated on every save

Note: assigned_analyst_id is a foreign key from the User table back onto itself. It lets an analyst's dashboard resolve "all investors assigned to me" with a single filtered query, without needing a separate assignment table.

6.3 Table: Portfolio (portfolios app)

Field	Type	Constraints / Notes
id	Integer	Primary key
user_id	Integer	Foreign key to User.id; the owning investor
name	String	Required
description	String	Optional
created_at	DateTime	Set on creation
updated_at	DateTime	Updated on every save

Relationship: one User has many Portfolios (one-to-many).

6.4 Table: Holding (portfolios app)

Field	Type	Constraints / Notes
id	Integer	Primary key

Field	Type	Constraints / Notes
portfolio_id	Integer	Foreign key to Portfolio.id
ticker	String	Stock symbol, e.g. AAPL; used to join against PriceHistory
quantity	Decimal	Number of shares held
buy_price	Decimal	Price per share at purchase
buy_date	Date	Date of purchase
risk_label	Enum	Low / Medium / High — written by the analytics app
risk_score	Float	Model confidence/probability behind risk_label
predicted_direction	Enum	Up / Down — written by the analytics app
prediction_confidence	Float	Model probability behind predicted_direction
last_predicted_at	DateTime	Timestamp of the most recent ML recomputation

Relationship: one Portfolio has many Holdings (one-to-many). The risk and forecast fields are read-only from the API's perspective — they are only ever written by Celery tasks in the analytics app, never by a user-facing write endpoint.

6.5 Table: PriceHistory (market_data app)

Field	Type	Constraints / Notes
id	Integer	Primary key
ticker	String	Stock symbol
date	Date	Trading date
open_price	Decimal	
close_price	Decimal	
high_price	Decimal	
low_price	Decimal	
volume	Integer	Shares traded that day

Note: PriceHistory is deliberately not linked to Holding by a foreign key. It is ticker-level market data shared by every holding (and every user) that references that ticker — attaching it to one Holding would incorrectly scope shared data to a single owner and would duplicate rows across every investor who holds the same stock. Instead, it is joined at query time on the shared ticker field. This is a normalization decision worth explaining in an interview.

6.6 Relationship Diagram (described)

User (1) — (many) Portfolio (1) — (many) Holding (many) — (matched by ticker) — PriceHistory. Additionally, User (1) — (many) User, via assigned_analyst_id, models the analyst-to-investor relationship.

6.7 Representative Join-Based Queries

- **Portfolio summary —** join Portfolio → Holding → the most recent PriceHistory row per ticker, to compute current total portfolio value and unrealized gain/loss.
- **Investor dashboard —** join User → Portfolio → Holding to aggregate total holdings and a risk-label distribution (count of Low/Medium/High) across all of a user's portfolios.
- **Analyst dashboard —** self-join on User (analyst → assigned investors via assigned_analyst_id), then join onward to Portfolio → Holding, to produce a consolidated view across every client assigned to that analyst.
- **Ticker trend query —** join Holding → PriceHistory on ticker, filtered by a date range, to render a price history series for a specific holding.

All of the above are expressed as Django ORM queries using select_related (for forward foreign keys such as Holding → Portfolio → User) and prefetch_related (for reverse relationships such as Portfolio → Holding), plus values()/annotate() for aggregation — never raw SQL.

7. Authentication & Authorization

7.1 Why a Custom User Model

Django's default User model has no concept of role or analyst assignment, and retrofitting those onto it later is painful because so much of the framework assumes the default model from the first migration. This project defines its own User model in the accounts app from the outset, with role and assigned_analyst_id as first-class fields, and configures it as the project's AUTH_USER_MODEL.

7.2 JWT Token Lifecycle

- **Registration —** a new account is created with role fixed to investor by default; only an admin can change a user's role afterward.
- **Login —** credentials are validated against the stored password hash; on success, the system issues an access token and a refresh token, each signed with its own secret key.
- **Authenticated requests —** the access token is sent as a Bearer token in the Authorization header; a custom authentication class decodes and validates it against JWT_SECRET_KEY and loads the corresponding User.
- **Refresh —** when the access token expires, the client sends the refresh token to a dedicated endpoint, which validates it against JWT_REFRESH_SECRET_KEY and issues a new access token.
- **Logout —** the refresh token is blacklisted/invalidated so it can no longer be exchanged for new access tokens.

7.3 Token Lifetimes

Access tokens are short-lived (15 minutes) to limit the damage window if one is ever leaked; refresh tokens last longer (7 days) so a user is not forced to log in daily. Using two separate signing secrets — one for access tokens, one for refresh tokens — means that even if one key were ever compromised, the other token type would still be cryptographically safe, which is a meaningful defense-in-depth decision worth calling out.

7.4 Authorization Model

Two layers of authorization are enforced on every protected endpoint:

- **Role-based access —** certain endpoints are gated entirely by role (for example, only admin may list all users or change roles).
- **Object-level access —** even where a role is allowed to hit an endpoint, the queryset itself is filtered so an investor only ever sees rows they own, and an analyst only ever sees rows belonging to their assigned investors. This is enforced at the queryset level, not just in the view's decision to allow or deny the request, which closes off insecure-direct-object-reference (IDOR) style access to another user's data by guessing an ID.

7.5 Custom Permission Classes

Permission class	Applies to	Rule
IsOwner	Portfolio, Holding	Request user must be the owning investor
IsAssignedAnalyst	Portfolio, Holding, risk/forecast views	Request user must be an analyst, and the resource's owner's assigned_analyst_id must equal the request user's id
IsAdminRole	User management, role assignment, system-wide views	Request user's role must be admin
IsOwnerOrAssignedAnalystOrAdmin	Most read endpoints	Combines the three rules above with OR logic for read access

8. Roles & Permission Matrix

Action	Investor	Analyst	Admin
Register / log in	Yes	Yes	Yes
View own portfolios & holdings	Yes	N/A	Yes
View assigned investors' portfolios	No	Yes	Yes
View all portfolios system-wide	No	No	Yes
Create / edit / delete own holdings	Yes	No	Yes
Trigger manual price refresh	No	Yes	Yes
Trigger ML risk/forecast recompute	No	Yes	Yes
Assign analysts / manage roles	No	No	Yes
View system-wide analytics	No	No	Yes

9. API Endpoint Reference

All endpoints are versioned under /api/ and, except registration and login, require a valid JWT access token. "Owner/Analyst/Admin" in the Allowed Roles column means the endpoint is reachable by any of the three, with the actual rows returned scoped per the permission rules in Section 7.

9.1 accounts

Method	Endpoint	Description	Allowed Roles
POST	/api/accounts/register/	Create a new investor account	Public
POST	/api/accounts/login/	Validate credentials, issue access + refresh token pair	Public
POST	/api/accounts/token/refresh/	Exchange a valid refresh token for a new access token	Public (valid refresh token required)
POST	/api/accounts/logout/	Blacklist the current refresh token	Any authenticated user
GET	/api/accounts/me/	Retrieve the logged-in user's profile	Any authenticated user
PATCH	/api/accounts/me/	Update the logged-in user's own profile	Any authenticated user
GET	/api/accounts/users/	List all users	Admin
PATCH	/api/accounts/users/{id}/	Update a user's role or assigned_analyst_id	Admin

9.2 portfolios

Method	Endpoint	Description	Allowed Roles
GET	/api/portfolios/	List portfolios visible to the requester (own / assigned / all)	Investor, Analyst, Admin
POST	/api/portfolios/	Create a new portfolio	Investor, Admin
GET	/api/portfolios/{id}/	Retrieve portfolio detail, including holdings	Owner, Assigned Analyst, Admin
PATCH	/api/portfolios/{id}/	Update portfolio name/description	Owner, Admin
DELETE	/api/portfolios/{id}/	Delete a portfolio	Owner, Admin
GET	/api/portfolios/{id}/summary/	Aggregated value and risk distribution for the portfolio	Owner, Assigned Analyst, Admin
GET	/api/portfolios/{id}/holdings/	List holdings within a portfolio	Owner, Assigned Analyst, Admin
POST	/api/portfolios/{id}/holdings/	Add a holding to a portfolio	Owner, Admin
GET	/api/holdings/{id}/	Retrieve a single holding, including risk/forecast fields	Owner, Assigned Analyst, Admin
PATCH	/api/holdings/{id}/	Update quantity / buy price / buy date	Owner, Admin
DELETE	/api/holdings/{id}/	Remove a holding	Owner, Admin

9.3 market_data

Method	Endpoint	Description	Allowed Roles
GET	/api/market/prices/{ticker}/	Latest known price for a ticker	Any authenticated user

Method	Endpoint	Description	Allowed Roles
GET	/api/market/prices/{ticker}/history/	Historical OHLCV series, filterable by date range	Any authenticated user
POST	/api/market/refresh/{ticker}/	Queue an async task to re-fetch data for one ticker	Analyst, Admin

9.4 analytics

Method	Endpoint	Description	Allowed Roles
GET	/api/analytics/holdings/{id}/risk/	Current risk label, score, and last computed time	Owner, Assigned Analyst, Admin
GET	/api/analytics/holdings/{id}/forecast/	Current predicted direction and confidence	Owner, Assigned Analyst, Admin
POST	/api/analytics/holdings/{id}/recompute/	Queue an async recomputation for one holding	Analyst, Admin
GET	/api/analytics/portfolio/{id}/risk-summary/	Portfolio-level risk breakdown	Owner, Assigned Analyst, Admin

10. Serializers Overview

Serializers are the contract between the Django ORM and any client, including the planned Flutter app. Each one is described below in terms of purpose, exposed fields, and validation behaviour — no implementation code, since the same description should hold regardless of exact syntax.

UserSerializer

Exposes id, username, email, role, assigned_analyst (a nested, minimal representation — id and username only), and date_joined. The password field is write-only and only accepted on creation; it is validated for minimum length and hashed before saving, never returned in a response. The role field is read-only for anyone except admin, enforced at the serializer level in addition to the permission class.

RegisterSerializer

Validates that username and email are unique, validates a password-confirmation pair, and creates a User with role defaulted to investor. Kept separate from UserSerializer so that registration validation rules never leak into profile-update logic.

LoginSerializer

Validates the submitted username/email and password against stored credentials and, on success, hands off to the token-issuing logic that produces the access/refresh pair.

PortfolioSerializer

List view returns id, name, description, created_at, and two computed fields: holdings_count and total_value (resolved via the Holding → PriceHistory join described in Section 6.7). Detail view additionally nests a list of HoldingSerializer objects for that portfolio, plus a minimal owner representation.

HoldingSerializer

Exposes id, ticker, quantity, buy_price, buy_date, a computed current_price (resolved by matching ticker against the latest PriceHistory row), and the ML output fields — risk_label, risk_score, predicted_direction, prediction_confidence, last_predicted_at. The ML fields are read-only at the serializer level: they can only ever be populated by the analytics Celery tasks, never by a client-supplied value, which prevents a user from spoofing their own risk score.

PriceHistorySerializer

Exposes ticker, date, and the four OHLC prices plus volume. Entirely read-only from the API's perspective; rows are only ever created by the market_data ingestion tasks.

RiskSummarySerializer

A response-only serializer (not tied to a single model) that shapes the output of the portfolio risk-summary endpoint: portfolio_id, total_value, a risk_distribution object (counts of Low/Medium/High across the portfolio's holdings), and average_confidence across predictions.

List vs. detail field strategy

List endpoints intentionally use lighter serializers with fewer fields to keep payloads small for a mobile client, while detail endpoints return the fully nested representation. This is paired with select_related/prefetch_related in the corresponding queryset so that returning the extra nested detail does not introduce N+1 query problems.

11. Celery & Redis Task Design

11.1 Redis's Three Roles

- **Celery broker —** Redis database 0 (CELERY_BROKER_URL) holds the queue of pending tasks that Celery workers pull from.
- **Celery result backend —** Redis database 1 (CELERY_RESULT_BACKEND) stores task results and state, kept in a separate logical database from the broker so a long results history never competes with, or gets flushed alongside, the live task queue.
- **Application cache —** the general-purpose REDIS_URL connection is used through Django's cache framework to cache expensive, read-heavy aggregation responses.

11.2 Scheduled and On-Demand Tasks

Task	Trigger	Responsibility
fetch_daily_prices	Celery Beat — daily, after market close	Pulls the latest OHLCV row for every tracked ticker and bulk-inserts into PriceHistory
refresh_single_ticker	On-demand — POST /api/market/refresh/{ticker}/	Fetches data for one ticker outside the daily schedule, for analysts/admins who need fresher data immediately

Task	Trigger	Responsibility
recompute_holding_risk	Celery Beat — nightly	Recomputes risk_label and risk_score for every holding using the latest engineered features
recompute_holding_forecast	Celery Beat — nightly	Recomputes predicted_direction and prediction_confidence for every holding
recompute_single_holding	On-demand — POST /api/analytics/holdings/{id}/recompute/	Recomputes risk and forecast for one holding immediately
retrain_models	Celery Beat — weekly/monthly	Retrains the Random Forest and Gradient Boosting models on accumulated data and records a new model_version

11.3 Caching Strategy

Portfolio summary and portfolio risk-summary responses are cached in Redis with a short time-to-live (around five minutes). The cache is explicitly invalidated whenever a holding is created, updated, or deleted, and whenever a recomputation task completes for a holding in that portfolio — so the TTL is a safety net, not the primary invalidation mechanism.

11.4 Rate Limiting

The manual refresh endpoint is throttled per user using a Redis-backed DRF throttle class, to prevent a single user from hammering the external market data provider through repeated on-demand refresh calls.

12. Machine Learning Pipeline

12.1 Data Sources

The pipeline draws on two tables: PriceHistory, for the raw OHLCV time series per ticker, and Holding, for position-level metadata such as buy price, quantity, and buy date.

12.2 Feature Engineering

A shared feature set is engineered per ticker/holding and reused by both models:

- Rolling volatility — standard deviation of daily returns over 30-day and 90-day windows
- Maximum drawdown over a trailing window
- Moving averages — 20-day and 50-day
- Relative Strength Index (RSI)
- Average trading volume trend
- Holding age — days since buy_date
- Unrealized gain/loss percentage relative to buy_price

12.3 Model 1 — Risk Classification

- Algorithm: Random Forest Classifier.
- Target: risk_label (Low / Medium / High), derived from historical volatility buckets used as training labels.
- Output written to Holding: risk_label and risk_score (the class probability of the predicted label).
- Why this algorithm: it performs well on engineered tabular features without large data requirements, trains quickly, and — importantly for this project — exposes feature importances, so the model's behaviour can be explained rather than treated as a black box.
- Baseline comparison: a Logistic Regression baseline is trained and evaluated alongside the Random Forest, so the choice of the more complex model is backed by a measured improvement rather than assumed.

12.4 Model 2 — Directional Forecast

- Algorithm: Gradient Boosting Classifier (XGBoost/LightGBM).
- Target: a binary label — whether the next trading day's close is Up or Down relative to the current close — built from the engineered technical-indicator features.
- Output written to Holding: predicted_direction and prediction_confidence (the predicted probability).
- Why direction, not price: framing the forecast as a directional classification problem, rather than attempting to predict an exact future price, is both easier to validate honestly and more credible to discuss in an interview than a precise price prediction, which even professional models rarely achieve reliably.

12.5 Training Methodology

Both models use a chronological (time-based) train/test split — the model is trained on an earlier window of data and evaluated on a later, held-out window — rather than a random shuffle split. This deliberately avoids lookahead bias, where a randomly-split model could be evaluated on data that "leaks" information from the future relative to its training set.

12.6 Evaluation Metrics

Model	Metrics tracked
Risk classifier (Random Forest)	Precision, recall, and F1 per class (Low/Medium/High); confusion matrix
Directional forecaster (Gradient Boosting)	Accuracy, F1, and ROC-AUC

12.7 Retraining & Versioning

Models are retrained on a schedule (weekly or monthly, via the retrain_models Celery Beat task) as new PriceHistory accumulates. Each trained model is tagged with a model_version, which is recorded alongside the predictions it produces, so any risk_label or predicted_direction on a Holding can be traced back to the exact model version that generated it.

13. Environment Configuration Reference

All environment-specific configuration is kept in a .env file, excluded from version control, and loaded at application startup. The table below documents every variable used by this project.

13.1 Database

Variable	Value in this project	Purpose
DATABASE_NAME	stock_risk_prediction_db	Name of the PostgreSQL database
DATABASE_USER	django_user	PostgreSQL role Django connects as
DATABASE_PASSWORD	(kept secret)	Password for that role; never committed to source control
DATABASE_HOST	localhost	Database host — a managed host in staging/production
DATABASE_PORT	5432	Standard PostgreSQL port

13.2 JWT

Variable	Purpose
JWT_SECRET_KEY	Signs and verifies access tokens; distinct from Django's own SECRET_KEY
JWT_REFRESH_SECRET_KEY	Signs and verifies refresh tokens using a separate key from the access token, so compromise of one token type does not compromise the other
ACCESS_TOKEN_LIFETIME_MINUTES	Set to 15 — how long an access token remains valid
REFRESH_TOKEN_LIFETIME_DAYS	Set to 7 — how long a refresh token remains valid before the user must log in again

13.3 Redis & Celery

Variable	Value in this project	Purpose
REDIS_URL	redis://localhost:6379/0	General application cache connection
CELERY_BROKER_URL	redis://localhost:6379/0	Redis database used as the Celery message broker
CELERY_RESULT_BACKEND	redis://localhost:6379/1	Separate Redis database used to store Celery task results

Note: .env is listed in .gitignore and never committed. Secrets differ per environment (local, staging, production) and are rotated independently of application code changes.

14. Project Directory Structure

The layout below is described conceptually, grouped by responsibility rather than as literal file paths:

- **config —** Django project settings, split by environment (base / development / production), and .env loading.
- **accounts —** the custom User model, authentication serializers and views, JWT issuing/refresh logic, and admin-facing user management.

- **portfolios —** Portfolio and Holding models, their serializers, viewsets, and object-level permission classes.
- **market_data —** the PriceHistory model, ingestion Celery tasks, and price-related read endpoints.
- **analytics —** feature engineering code, model training/retraining scripts, the Celery tasks that write risk/forecast fields, and the read-only analytics endpoints.
- **core —** shared permission classes, pagination classes, exception handling, and throttle classes used across apps.
- **requirements —** dependency lists, separated per environment.
- **manage.py —** the standard Django management entry point.

15. Security & Non-Functional Considerations

- Passwords are hashed using Django's standard password hasher even though the User model is custom — the hashing/validation behaviour is reused, only the model shape changes.
- JWTs are intended for secure client-side storage (e.g., Flutter secure storage in the planned mobile client), never stored in a way equivalent to insecure browser local storage.
- Object-level permission checks are applied at the queryset level, not only as a view-level allow/deny decision, closing off insecure-direct-object-reference (IDOR) access to another user's data.
- Login and registration endpoints are throttled to reduce the effectiveness of brute-force attempts.
- CORS is scoped to known client origins (local development and the future Flutter app's origin/scheme), not left open.
- Database indexes are placed on PriceHistory(ticker, date), Holding(portfolio_id), and Portfolio(user_id) to keep the join-heavy summary queries fast as data grows.
- All list endpoints are paginated to bound response size, which matters both for API performance and for a mobile client's data usage.
- Celery task failures are logged with enough context (task name, arguments, ticker/holding id) to be debuggable without reproducing the failure manually.

16. Future Scope & Flutter Integration Readiness

Because the backend is API-first — every capability is already reachable through a JWT-authenticated REST endpoint with a well-defined serializer — a Flutter client can be built directly against the existing API surface without any backend changes. The planned Flutter app would consist of: login/registration screens against the existing accounts endpoints, a portfolio dashboard consuming the summary endpoint, and a holding detail screen visualising the risk and forecast fields already returned by the analytics endpoints.

Further extensions under consideration, in rough priority order:

- Push notifications (e.g., via Firebase Cloud Messaging) triggered from Celery when a holding's risk_label changes or a forecast crosses a confidence threshold.
- WebSocket or Django Channels support for live price updates, rather than polling.
- Sector-level analytics and portfolio diversification scoring.

- Portfolio rebalancing suggestions derived from the existing risk scores.
- An LSTM-based forecasting model as a stretch goal, positioned explicitly as a comparison against the existing Gradient Boosting baseline rather than a replacement, once the simpler model's performance is well understood.
- Containerisation (Docker/Docker Compose) if and when the project moves toward a shared or production-style deployment.

17. Machine Learning Dataset Source

The historical price data used to train and validate both machine learning models was sourced from the following public dataset:

Dataset: S&P 500 Stocks — Daily Historical Data (10 Years), Kaggle

17.1 Why It Maps Directly onto the PriceHistory Table

The documented schema is:

ticker | date | open_price | close_price | high_price | low_price | volume

This dataset is historical OHLCV data for all current S&P 500 companies covering 2015 to 2025, and structurally it is a collection of individual CSV files, where each file represents a specific company identified by its ticker symbol — for example AAPL.csv, MSFT.csv — with columns for Date, Open, High, Low, and Close.

That is essentially a one-file-per-ticker version of the PriceHistory table: the CSVs are concatenated, a ticker column is added from each filename, and the result is loaded via the ORM (or a management command / seed script) exactly as the ingestion task in Section 11 describes — just as a one-time historical backfill instead of a live daily pull.

17.2 Why 10 Years of Daily Data Specifically Fits This ML Design

- The feature set (30/90-day rolling volatility, 20/50-day moving averages, RSI) needs a solid amount of trailing history per ticker before the first row is even usable — 10 years easily covers that with room to spare.
- The time-based train/test split (train on older data, test on a held-out recent window) needs enough calendar span to have a meaningful "recent" holdout — a couple of years is thin for that; 10 years is comfortable.
- Multiple tickers (S&P 500 breadth) let the Random Forest risk classifier learn from cross-sectional variation in volatility/drawdown patterns, not just one stock's idiosyncratic history.

18. Appendix — Sample Interaction Flow

The following walks through a representative end-to-end flow in narrative form, tying together the pieces described above:

- An investor registers an account and logs in, receiving an access token and a refresh token.

- The investor creates a new portfolio, then adds a holding by specifying a ticker, quantity, buy price, and buy date.
- If that ticker has no recent PriceHistory rows, the system queues an asynchronous ingestion task rather than making the investor wait on an external API call.
- That night, the scheduled recompute_holding_risk and recompute_holding_forecast tasks run, engineer features from the now-available PriceHistory, and write a risk_label, risk_score, predicted_direction, and prediction_confidence onto the holding.
- The investor requests their portfolio summary; if a cached response exists in Redis and is still fresh, it is returned immediately, otherwise the summary is computed via the Portfolio → Holding → PriceHistory join and then cached.
- The analyst assigned to this investor (via assigned_analyst_id) logs in separately and views the same portfolio in a read-only capacity, made possible entirely by the IsAssignedAnalyst permission class rather than any special-cased view logic.
- When the investor's access token expires, their client silently exchanges the refresh token for a new access token, with no interruption to the session.